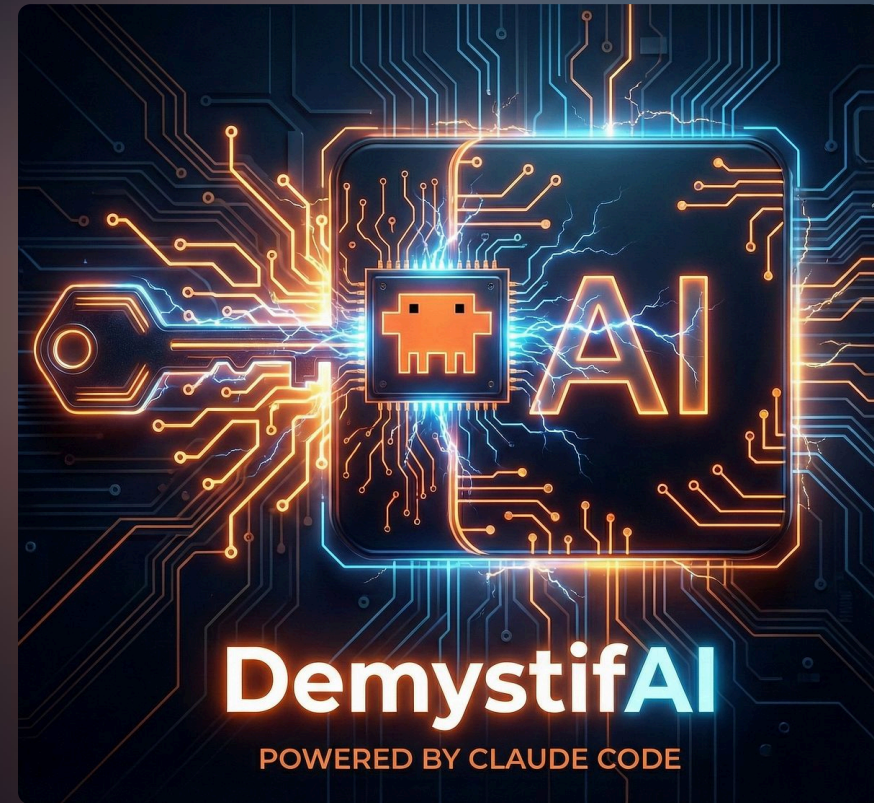


Claude Code: From Zero to Expert

Workshop #8: MCP's, Standards & Scaling

Presented by: Tomasz Guściora

For Software Mind



Tomasz Guściora - Bio

- 13+ years of experience in data analytics
- Analytical consultant with successful projects at numerous international financial institutions
- AI and Large Language Model (LLM) practitioner and trainer
- Creator of [DemystifAI.blog](https://demystifai.blog)
- Claude Code Trainings: [MasterClaudeCode.now](https://masterclaudecode.now)
- Substack Blog: [DemystifAI.substack.com](https://demystifai.substack.com)
- [LinkedIn](#)
- Email: Tomasz@gusciora.pl

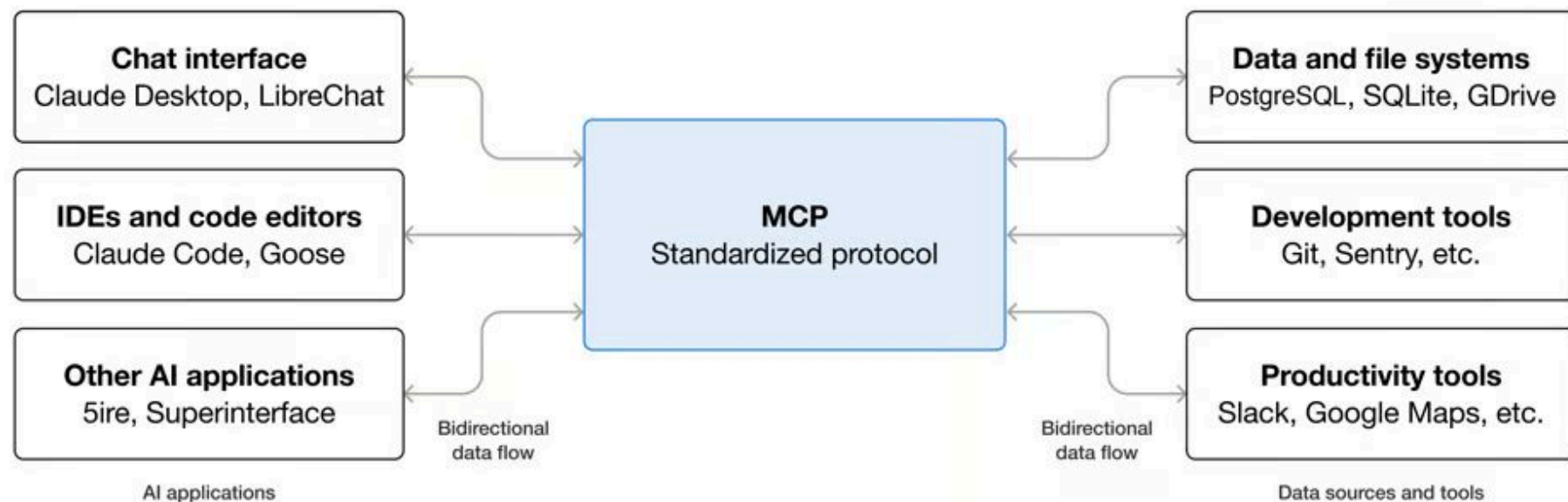


What is Model Context Protocol?

MCP: The USB-C for AI Connections

The Model Context Protocol (MCP) is an open standard, pioneered by Anthropic and supported by other industry leaders. It aims to standardize how AI models integrate with diverse data sources and external tools.

Instead of building separate integrations for each model and database, MCP allows developers to create a single, universal "connector" that works everywhere, vastly simplifying AI harness development and deployment.



Source: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro>

MCP: Navigating Trade-offs and Complexity

Implementing MCP introduces fundamental trade-offs: balancing enhanced business value from wide-system tool access with stringent security and capability requirements. Exposing too many tools through MCP can lead to context overload for the LLM.



LLM Confusion

Difficulty in discerning relevant information from excessive tool definitions.



Hallucination

Increased tendency to generate incorrect or fabricated responses due to context noise.



Higher Error Rates

Elevated likelihood of operational failures and suboptimal AI performance.

While MCP solves the challenge of AI connectivity, it shifts the focus to intelligently managing MCP selections to avoid these risks.

MCP Actors: The Core Components



MCP Host

The central application or orchestrator, typically your primary service (Claude Code).



MCP Client

Connects the Host to external services. A Host can summon multiple Clients, creating a one-to-many relationship.



MCP Server

Provides access to resources, tools, and prompts, offering standardized functionalities. Examples include GitHub API, Stripe Payments, or Supabase DB.

A special case is the **local MCP server**, which requires installation and configuration to share local filesystems (like standard I/O).

MCP: Evolving Connection Standards

MCP has explored various communication protocols to connect models with external tools, each presenting a distinct set of trade-offs in terms of performance, simplicity, and maintainability.

Protocol	Pros	Cons
STDIO	Fast and secure (within one machine), optimal for single developer environments or initial development phases.	Requires specific language dependencies (e.g., Python, Node.js) to be installed on each user's machine.
SSE (Server-Sent Events)	Deprecated as of March 26, 2025	Introduced significant connection management overhead due to a dual-endpoint architecture (separate /sse and /sse/messages).
Streamable HTTPS + JSON-RPC	Utilizes a single endpoint (/mcp), supports bi-directional communication, and is an established enterprise-grade standard.	alternative to WebSocket

The MCP Ecosystem: Core Concepts

Within the Model Context Protocol (MCP), several core concepts define how AI agents interact with external systems. Understanding these concepts is crucial for effective integration and management of AI tools.



TOOL

Commands that trigger specific actions or operations within an external system.

- **Example:** Instead of passively viewing, a tool might be designed to **calculate** a result using a calculator or **send** an email via a mail client.
- **Key:** Tools typically involve **side-effects** that alter the state of the external system, such as sending a message, which cannot be easily undone.



RESOURCE

Static or dynamic data sources that provide information for an AI agent to consume.

- **Example:** To write a report, an agent might need to **read** logs from the past week. It consumes this information without modifying it, gaining knowledge for its task.
- **Key:** Resources are typically addressed via URIs, similar to web pages (e.g., `logs://error-log`), allowing the agent to request information at a specific address.



PROMPT

Pre-defined templates or instructions that guide the AI agent's behavior and interaction with tools or resources.

- **Example:** Instead of repeatedly instructing an agent on error analysis steps ("first check date, then module, then solution"), a structured `analyze_error` prompt can be provided.
- **Key:** Prompts allow the server (tool creator) to enforce best practices and standardize how the agent utilizes available services, ensuring consistent and effective operation.

The MCP Registry - catalogue of MCP's

The [Official MCP Registry](#) serves as the central catalogue for Model Context Protocol definitions, offering both public and private options for discovering and managing MCPs.

While inclusion in the registry streamlines discovery, it is crucial to note that **being listed in the MCP Registry does not automatically guarantee the quality or security of the MCP code**. Rigorous verification is still essential.



Audit Source Code

Thoroughly review the MCP's underlying source code before connection or execution to identify potential vulnerabilities or malicious intent.



Verify Namespaces & Authors

Confirm the authenticity and reputation of the MCP's creators and associated namespaces to prevent impersonation and supply chain attacks.



Sandbox Execution

Launch MCPs in hardened, isolated containers instead of granting direct root access to servers, limiting potential damage from compromised code.

Excercise (20 mins): MCP Pizza Order Tracking

Let's consider a practical example: building a Model Context Protocol (MCP) server for a pizza application. This demonstrates how an AI can manage real-world service interactions from ordering to delivery.

**Order Placement**

Customers can select menu items and specify a delivery address, initiating a new order.

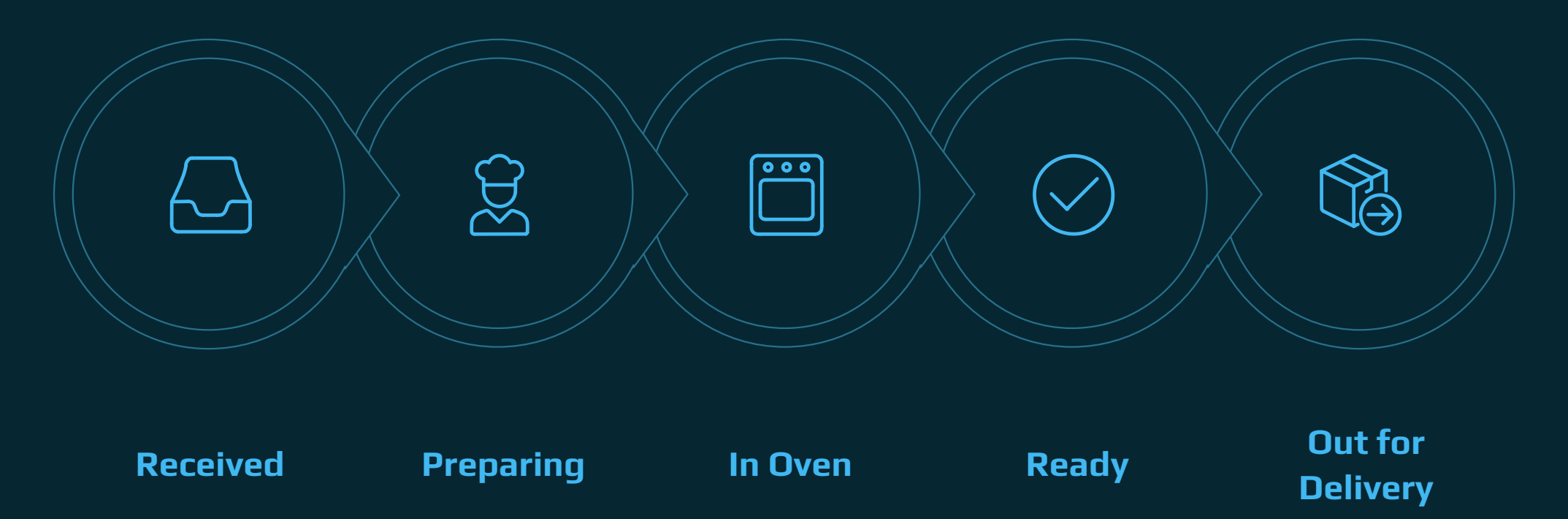
**Real-time Tracking**

Monitor the current status of an order as it progresses through the delivery pipeline.

**Conditional Cancellation**

Customers retain the ability to cancel an order, but only up to a predefined stage (e.g., before dispatch).

Pizza Order Lifecycle



This sequential workflow ensures transparency from the moment an order is placed until it reaches the customer's door.

Test with Claude: "Order a Lone Star Heat to 1 Main St, Austin for The Terminator. Use Pizza Order Tracking MCP and tell me it's status"

Proprietary MCP Ideas

Here are examples of proprietary Model Context Protocol (MCP) servers and the critical tools and resources they can expose, enabling powerful AI integrations within an enterprise environment.

#	Server	Key tools / resources
1	Service catalog	<code>find_service</code> , <code>get_service</code> (owner, on-call, SLOs, consumers, runbooks)
2	Engineering standards	<code>get_standard(topic, lang)</code> , <code>check_compliance(diff)</code> → rule IDs
3	CI diagnostics	<code>get_failed_jobs</code> , <code>get_log_excerpt</code> , <code>classify_failure</code> , <code>get_flake_history</code>
4	Internal API discovery	<code>search_apis</code> , <code>get_openapi</code> , <code>get_client_snippet</code>
5	Database schema (read-only)	<code>describe_table</code> , <code>list_indexes</code> , <code>explain_query</code> (against a dev replica)
6	Feature flags	<code>get_flag_state</code> , <code>propose_flag_change</code> (opens a change request, doesn't write)
7	Deployment	<code>get_deploy_status</code> , <code>plan_deploy</code> , <code>request_deploy</code> (gated)
8	Incidents	<code>get_active_incidents</code> , <code>get_timeline</code> , <code>find_similar_incidents</code>
9	Architecture / ADRs	<code>search_adrs</code> , <code>get_boundary_rules(module)</code>
10	Dependency ownership	<code>who_owns_package</code> , <code>get_license_verdict</code> , <code>get_approved_alternatives</code>
11	Infrastructure (mostly read)	<code>get_k8s_workload</code> , <code>get_cost_by_service</code>
12	Company docs search	<code>search_docs(query, space)</code> ; only if skill reference files aren't enough

These examples highlight the potential for MCP to streamline operations, enhance developer productivity, and improve decision-making by giving AI direct, controlled access to internal systems.

CLI vs. MCP: Choosing the Right Interface

Selecting between a Command Line Interface (CLI) and the Model Context Protocol (MCP) depends on the specific interaction needs of your AI application. Both offer distinct advantages and trade-offs.

¹ Command Line Interface (CLI)

- **Direct Control**
- **Scriptable**
- **Developer Familiarity**
- **No Protocol Overhead**
- **Limited AI Understanding**
- **Poor Discoverability**
- **Local Dependency**

² Model Context Protocol (MCP)

- **AI-Native Design**
- **Enhanced Discoverability**
- **Complex Ecosystems**
- **Type-Safe Interactions**
- **Requires Implementation**
- **Protocol Adherence**
- **Abstraction Layer**

Declaring MCP in Claude Code

Standardized connections to external tools and data.

Further MCP registries:

- <https://cursor.com/docs/context/mcp/directory>
- <https://github.com/mcp>

Configuration (.mcp.json in project root):

```
```json
{
 "mcpServers": {
 "github": {
 "type": "http",
 "url": "https://api.githubcopilot.com/mcp/",
 "headers": { "Authorization": "Bearer ${GITHUB_TOKEN}" }
 }
 }
}
```
```

Connecting Servers

HTTP (Recommended)

Cloud-based services

```
claude mcp add --transport http <name> <url>
```

Stdio (Local)

Direct system access/custom scripts

```
claude mcp add --transport stdio <name> -- <command>
```

Control Your Environment

Add: ``claude mcp add [options]``

List: ``claude mcp list`` (View active connections)

Inspect: ``claude mcp get``

Remove: ``claude mcp remove``

Status: ``/mcp`` (Inside Claude Code interface)

Exercise (10 mins): Querying the Claude Code Changelog

Let's simulate a scenario where Claude needs to access an internal knowledge base to find the latest changes in Claude Code. This exercise demonstrates how to integrate a new MCP server and query it for specific information.



Add 'context7' MCP Server

First, we'll configure Claude to connect to a hypothetical 'context7' MCP server, which is designed to expose internal documentation and change logs via HTTP.

```
claude mcp add --transport http context7  
https://api.context7.com/mcp/docs
```



Query Latest Claude Code Changelog

Now, use the 'context7' MCP to ask for the most recent changelog entries for Claude Code. This simulates an AI agent retrieving specific, dynamic information from an internal system.

Use context7 to tell me what are the latest changelog sessions

This exercise highlights how MCP enables Claude to seamlessly interact with proprietary information sources, acting as an intelligent interface to complex data ecosystems.

Expected Outcome: Claude should return a summary or a list of the most recent changes to Claude Code, as if retrieving it directly from an internal documentation system.

The MCP Ecosystem

Development

- GitHub
- GitLab
- Linear
- Sentry

Data

- PostgreSQL
- Snowflake
- Google BigQuery

Productivity

- Notion
- Slack
- Asana
- Google Drive

Infrastructure

- AWS
- Cloudflare
- Vercel

Scopes & Permissions

Scopes

- **Project:** Specific to current directory (.mcp.json)
- **User:** Global availability

Security

- **Allowlist:** Explicitly permit servers
- **Denylist:** Block specific URLs/commands
-

Real Talk – Community Gotchas

Research from practice:

Context Amnesia

Compression causes Claude to ignore CLAUDE.md

Test Gaslighting

Claude modifying tests to pass broken code

Wildcard Fail

mcp__server__* permissions are unreliable

Silent Copying

Accidental .env secret leakage without hooks

Prompts are interpreted at runtime by an LLM that can be convinced otherwise. You need something deterministic.

Key Considerations for MCP Integration

Successful integration of Model Context Protocol (MCP) requires attention to three fundamental areas that ensure efficiency, reliability, and safety.



Discovery & Routing

Enabling agents to effectively **discover and route** through available tools and functionalities is crucial. This involves planning for **progressive disclosure** of capabilities and optimizing **context navigation** to leverage the right resources at the right time.



Execution & Reliability

Robust execution demands rigorous **request validation**, adherence to **strong types**, comprehensive **error handling**, and resilient **retries**. These measures ensure that interactions are predictable and minimize failures in dynamic environments.



Security & Privileges

Managing **privileges** and safeguarding against potential **vulnerabilities** is paramount. Implementing stringent access controls and auditing mechanisms protects against unauthorized actions and maintains the integrity of the ecosystem.

MCP Gateway Concept

The **MCP Gateway** (sometimes called a Hub) acts as an intermediary layer, much like an air traffic control tower. From an engineering perspective, it offers powerful capabilities:



Central Authorization

The client (Agent) authenticates once with the Gateway, which then manages complex keys for external services.



Monitoring & Auditing

Every request and response is visible in one place, providing comprehensive monitoring and audit trails for all activity.



Security

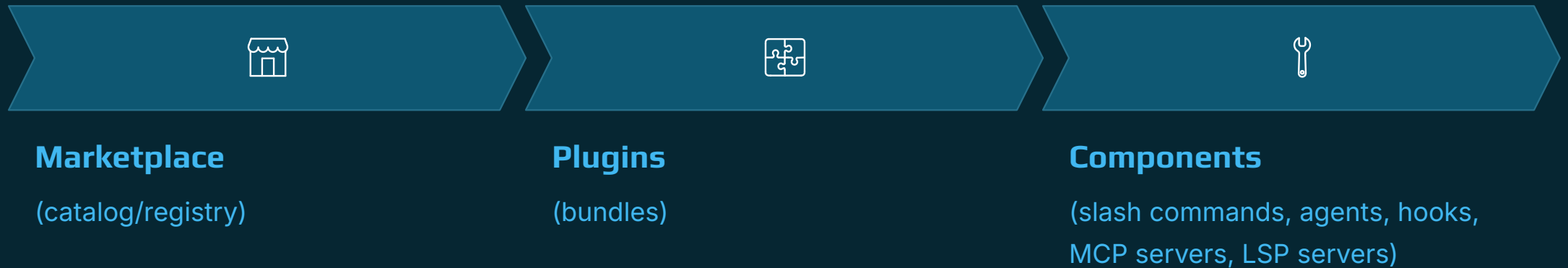
If an anomaly is detected (e.g., prompt injection or data leak), tool access can be instantly revoked for the entire organization without updating each Agent.

Source: <https://martinfowler.com/articles/gateway-pattern.html>

Claude Code Plugin Marketplace - How It Works

The plugin system (introduced in Claude Code ~2.0.13) turns CC from a standalone tool into an extensible platform. Think of it like package managers for your AI coding assistant.

Mental Model



A marketplace is a catalog of plugins someone has created and shared. Using one is two steps: first add the marketplace (registers the catalog), then browse and install individual plugins. No plugins are installed just by adding a marketplace.

Plugin entry point

...

/plugin # go to plugin marketplace

...

Plugins **Discover** Installed Marketplaces (+/- or tab to cycle)

Discover plugins (1/50)

🔍 Search...

- › ○ frontend-design · claude-plugins-official · 141.2K installs
Create distinctive, production-grade frontend interfaces ...
- context7 · claude-plugins-official [Community Managed] · 93.5K installs
Upstash Context7 MCP server for up-to-date documentation ...
- code-review · claude-plugins-official · 68.6K installs
Automated code review for pull requests using multiple sp...
- github · claude-plugins-official · 65.6K installs
Official GitHub MCP server for repository management. Cre...
- feature-dev · claude-plugins-official · 63.3K installs
Comprehensive feature development workflow with specializ...
- ↓ more below

Installation & Usage Checklist

Installation Steps:

Step 1 — Add marketplace

```
/plugin marketplace add  
https://bitbucket.org/org/repo.git
```

Step 2 — Install plugin:

```
/plugin install my-plugin@my-marketplace
```

Alternative step 2 — Enable in project
(.claude/settings.json):

```
```json  
{
 "enabledPlugins": {
 "my-plugin@my-marketplace": true
 }
}
```
```

Other Useful Commands:

```
```bash  
/plugin marketplace update my-marketplace
pull latest
/plugin list
see installed
/plugin validate .
check manifests
```
```

Pre-Publishing Checklist:

- marketplace.json at repo root .claude-plugin/
- plugin.json inside each plugin's .claude-plugin/
- Source paths start with ./ (not bare .)
- No agents, commands, category fields in plugin.json
- Hook scripts are executable and use \${CLAUDE_PLUGIN_ROOT}
- /plugin validate . passes clean
- /plugin validate ./my-plugin/ passes clean
- Plugin installed and enabled in target project's settings.json

Hooks Configuration & Validation

Hooks in plugin.json:

```
```json
"hooks": {
 "PreToolUse": [
 {
 "matcher": "Bash | Write | Edit",
 "hooks": [
 {
 "type": "command",
 "command":
"${CLAUDE_PLUGIN_ROOT}/hooks/my-hook.py"
 }
]
 }
],
 "PostToolUse": [...],
 "Stop": [...],
 "UserPromptSubmit": [...]
}
```
```

Hook Events:

- PreToolUse: Before a tool executes
- PostToolUse: After a tool executes
- UserPromptSubmit: When user sends a message
- Stop: When session ends

Tips:

- Use matcher to filter by tool name (pipe-separated regex)
- Use `${CLAUDE_PLUGIN_ROOT}` for paths — resolves to plugin install location
- Hook type: "command" runs a script (.py, .sh)

Validation Commands:

```
```bash
/plugin validate ./my-plugin/
validate plugin

/plugin validate .
validate marketplace (repo root)
```
```

Common Validation Errors:

- "No manifest found in directory" → Use plugin root dir, not .claude-plugin/
- "plugins.0.source: Invalid input" → Change "." to "./"
- "agents: Invalid input" → Remove agents field — auto-discovered
- "Unrecognized key: category" → Remove it — marketplace-only field
- "No marketplace description" → Add metadata object

Marketplace & Plugin Configuration

marketplace.json — Required Fields:

```
```json
{
 "name": "my-marketplace",
 "owner": {
 "name": "Your Name"
 },
 "metadata": {
 "description": "What this marketplace offers"
 },
 "plugins": [
 {
 "name": "my-plugin",
 "source": "./my-plugin",
 "description": "What the plugin does",
 "version": "1.0.0",
 "keywords": ["tag1", "tag2"],
 "author": {
 "name": "Your Name"
 }
 }
]
}
```
```

plugin.json — Keep It Minimal:

```
```json
{
 "name": "my-plugin",
 "description": "Brief plugin description",
 "version": "1.0.0",
 "author": { "name": "Your Name" },
 "license": "MIT",
 "keywords": ["dev", "tools"],
 "hooks": { }
}
```
```

Important Notes:

- Do NOT use: category, tags, strict in marketplace.json
- Do NOT use: agents, commands, category, skills in plugin.json — these cause validation errors
- Source paths must start with ./ (not bare .)
- Directories are auto-discovered

Plugin Marketplace Architecture

A plugin marketplace is a **git repository** that distributes Claude Code plugins.

Structure:

- Marketplace repo (git)
- `.claude-plugin/marketplace.json` ← registry
- Points to plugin directories ← actual plugins

Workflow:

1. Add marketplace
2. Browse plugins
3. Install
4. Enable

Folder Structure:

```
my-marketplace-repo/
├── .claude-plugin/
│   └── marketplace.json    ← REQUIRED at repo
root
|
├── my-plugin/              ← plugin directory
│   ├── .claude-plugin/
│   │   └── plugin.json    ← plugin manifest
│   ├── agents/            ← auto-discovered
│   ├── commands/          ← auto-discovered
│   ├── skills/             ← auto-discovered
│   ├── hooks/              ← auto-discovered
│   └── rules/              ← auto-discovered
|
└── another-plugin/         ← multiple plugins per
marketplace
    ├── .claude-plugin/
    │   └── plugin.json
```

Key Rules:

- `marketplace.json` must be at repo root `.claude-plugin/`
- `plugin.json` must be inside each plugin's `.claude-plugin/`
- `agents/`, `commands/`, `skills/`, `hooks/`, `rules/` are auto-discovered — do NOT declare them in `plugin.json`

Homework Assignment: Explore the MCP & Plugin's

To prepare for our next session, immerse yourself in the MCP Registry and identify practical applications.

- **Identify 3 MCPs:** Select three Micro-Credential Providers (MCPs) from the registry that you find interesting.
- **Identify 3 Plugins:** Choose three plugins that could enhance your current work scenarios or project workflows.
- **Outline Use Cases:** For each MCP and plugin, describe a possible use-case relevant to your daily tasks or future projects.

Let's discuss at the start of tomorrow's session.

Thank you for today!

Questions?

